GRAND LUXURY HOTEL & RESORT
Management System

A Desktop Reservation, Billing & Web-Portal Application

PROJECT REPORT

Java (Swing) Desktop Application | Auto-Generated HTML Presentation Portal

Version 2.0 • 2026

ACKNOWLEDGEMENT

This report documents the design, implementation and testing of the Grand Luxury Hotel & Resort Management System, a single-file Java Swing application paired with an auto-generated HTML presentation portal. The material draws directly on the accompanying source files, `HotelManagement.java` and `hotel.html`, and is organised in the conventional structure of a software project report: background and objectives, analysis, requirements, design, implementation, testing, results, and conclusion.

List of Figures

List of Tables

ABSTRACT

The Grand Luxury Hotel & Resort Management System is a single-file Java desktop application built with Swing that digitises the day-to-day front-desk operations of a small hotel. The system allows staff to record new guest reservations, calculate room charges automatically based on room category and length of stay, manage check-in and check-out status, browse a searchable table of all bookings, and view a live operational dashboard summarising occupancy and revenue. A custom-painted user interface layer (gradient headers, rounded buttons, card-style panels) gives the application a modern, polished look without relying on any external UI framework.

In addition to the desktop client, the application can generate a companion **hotel.html** file: a responsive, browser-viewable presentation of the hotel's room categories and pricing, intended for portfolio, lab-submission or public-facing display purposes. The project also includes an optional integration point for MongoDB, so that the in-memory booking list used during a session can, in a future iteration, be replaced by a persistent database-backed store without redesigning the rest of the application.

This report documents the problem this project addresses, the requirements gathered for it, the system and database design, the implementation details of each module, the testing carried out, and directions for future enhancement.

TABLE OF CONTENTS

CHAPTER 1

Introduction

1.1 Background

Small and mid-sized hotels frequently rely on paper registers or generic spreadsheets to record guest reservations, track which rooms are occupied, and calculate bills at check-out. This approach is slow, error-prone, and makes it difficult to answer simple operational questions — how many rooms are occupied right now, what is today's revenue, or which guests are due to check out — without manually scanning through records.

The Grand Luxury Hotel & Resort Management System was built to address exactly this gap for a small property with three room categories (Standard, Deluxe and Executive Suite). It is implemented as a single self-contained Java file so that it can be compiled and run with nothing more than a standard JDK installation, making it well suited to academic demonstration as well as light real-world use.

1.2 Problem Statement

Front-desk staff need a fast, visual, and accurate way to: (a) register a new booking and immediately see the computed price, (b) move a booking through its natural lifecycle from “Booked” to “Checked-In” to “Checked-Out”, (c) produce a readable, itemised bill including tax and service charge at the time a guest leaves, and (d) get an at-a-glance summary of how the hotel is performing. A manual or spreadsheet-based process satisfies none of these needs well.

1.3 Objectives

- Provide a guided booking form that validates guest details and computes the total price in real time.
- Maintain a single in-memory list of bookings that both the table view and the dashboard read from, keeping all views consistent.
- Support the check-in / check-out workflow with clear status colour-coding.
- Generate an itemised invoice that adds 12% GST and a 5% service charge on top of the room charge.
- Summarise operational data (total bookings, guests currently checked in, total revenue) on a dashboard.
- Offer an optional MongoDB persistence hook so the same codebase can be pointed at a real database with minimal change.
- Auto-generate a polished, responsive HTML page describing the hotel's rooms for web / portfolio presentation.

1.4 Scope of the Project

The current version manages a single hotel property with three fixed room categories and keeps booking data in memory for the duration of a session (with sample data pre-loaded for demonstration). Persistent storage, multi-property support, user authentication, and payment-gateway integration are treated as future enhancements and discussed in Chapter 9. The scope of this report covers the desktop application's four functional tabs — Room Booking, Check-In / Check-Out, All Records, and Dashboard — together with the HTML export feature.

CHAPTER 2

System Analysis

2.1 Existing System

In the manual/legacy scenario, booking details are written into a register or a generic spreadsheet. Price calculation is done by hand using a rate card, invoices are typed separately (often with copy-paste errors), and there is no single place where a manager can see occupancy or revenue without compiling numbers from multiple sources. Searching for a specific booking means scrolling through rows, and there is no consistent, colour-coded way to see which guests are currently in-house.

2.2 Proposed System

The proposed system centralises every booking as a structured **Booking** object held in a shared list. Every screen — the booking form, the records table and the dashboard — reads from and writes to this same list, so the data is always consistent. Price calculation, ID assignment, date stamping and status transitions are all handled in code, removing manual arithmetic and typing errors. A single click regenerates a shareable HTML summary of the hotel's offerings.

Table 2.1 — Existing vs Proposed System

Existing System	Proposed System
Manual arithmetic for pricing	Automatic price calculation from room rate × days
Paper/notebook records, easy to lose	Structured, searchable in-memory records table
No consolidated view of performance	Live dashboard: bookings, occupancy, revenue
Invoices typed by hand	Auto-generated itemised invoice with GST + service charge
No web-facing presentation	One-click HTML portal export

2.3 Feasibility Study

2.3.1 Technical Feasibility

The system uses only the standard Java SE libraries (Swing, AWT, java.time, java.io) that ship with any JDK, so no external dependency is mandatory to run the core application. MongoDB support is detected at runtime via reflection (`Class.forName`) and is used only if the driver happens to be on the classpath, so the technical bar to run the project is very low.

2.3.2 Economic Feasibility

Because the JDK, Swing, and the optional MongoDB Community driver are all free and open-source, the project has effectively zero licensing cost. The only ongoing cost in a production deployment would be hosting for a persistent database, which is optional.

2.3.3 Operational Feasibility

The interface groups tasks into four clearly labelled tabs, uses colour to communicate booking status, and validates input before it is accepted, so front-desk staff with basic computer literacy can operate it after a short walkthrough.

CHAPTER 3

Requirement Specification

3.1 Hardware Requirements

Component	Requirement
Processor	Any x86 / x64 processor capable of running a modern JVM (dual-core recommended)
RAM	2 GB minimum, 4 GB recommended
Storage	50 MB free space (application + generated HTML file)
Display	1024 × 768 or higher, for comfortable use of the Swing interface

3.2 Software Requirements

Component	Requirement
Language / Runtime	Java SE (JDK) 8 or later
GUI Toolkit	Java Swing / AWT (bundled with the JDK)
Operating System	Windows, macOS, or Linux — anything with a JVM
Optional Database	MongoDB (Community Server) + MongoDB Java Driver, for persistence
Browser	Any modern browser, to view the generated hotel.html portal

3.3 Functional Requirements

- FR-1: The system shall let staff enter guest name, phone number, room category and stay duration to create a booking.
- FR-2: The system shall calculate the estimated total price whenever the room category or number of days changes.
- FR-3: The system shall reject a booking if the guest name, phone number, or a valid positive number of days is missing.
- FR-4: The system shall assign a unique, auto-incrementing booking ID and room number to every new booking.
- FR-5: The system shall allow staff to search a booking by ID and update its status to Checked-In or Checked-Out.
- FR-6: The system shall generate an itemised bill including 12% GST and 5% service charge on check-out.
- FR-7: The system shall display all bookings in a sortable, colour-coded table.

- FR-8: The system shall show live counts of total bookings, currently checked-in guests, and total revenue.
- FR-9: The system shall export a static HTML page describing the hotel's room categories and pricing on demand.

3.4 Non-Functional Requirements

- Usability: the interface should use consistent colour semantics (green = success, red = danger/checked-out, blue = informational) so status is understandable at a glance.
- Performance: all operations (booking, search, table refresh) should complete in well under one second for realistic booking volumes, since data is held in memory.
- Portability: the application should run unmodified on any operating system with a compatible JVM.
- Maintainability: UI, data model, and business logic are kept in clearly separated inner classes/methods to simplify future extension.
- Extensibility: the MongoDB hook is designed so persistence can be added without reworking the UI layer.

3.5 Assumptions and Constraints

The current implementation makes a number of simplifying assumptions that a production deployment would need to revisit:

- Room numbers are assigned sequentially (100 + running booking count) rather than drawn from a fixed, finite room inventory, so the model does not currently prevent double-booking a physical room.
- All monetary values are in Indian Rupees (■) and tax rates (12% GST, 5% service charge) are hard-coded rather than configurable.
- Booking data lives only in memory for the duration of one run; restarting the application resets the list to the two sample bookings.
- There is a single implicit user role — any user of the desktop application has full access to every tab and action.
- The HTML export always reflects the three fixed room categories defined in the source; adding a new category requires a code change on both the Swing form and the HTML template.

CHAPTER 4

System Design

4.1 System Architecture

The application follows a simple layered architecture. The presentation layer (Swing components) never manipulates data directly; instead it calls into a small set of business logic methods (`calculatePrice`, `handleBooking`, `generateBillText`) which in turn operate on the shared in-memory `bookingList`. A persistence/export layer sits below that and is responsible for the optional MongoDB hook and the static HTML export.

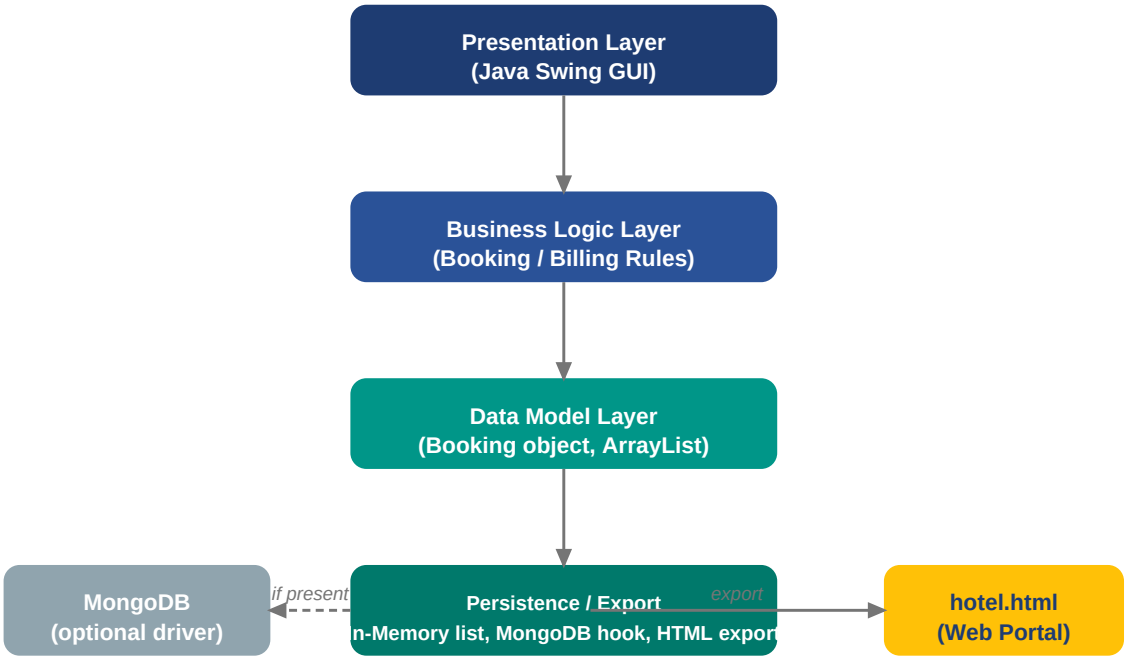

Fig. 4.1 — Layered architecture of the application

4.2 Data Flow Diagram (Level 0)

The Level-0 DFD below treats the whole application as a single process. Guests interact indirectly through front-desk staff, who submit booking requests and receive invoices and status updates in return; the process reads and writes booking records to the in-memory store.

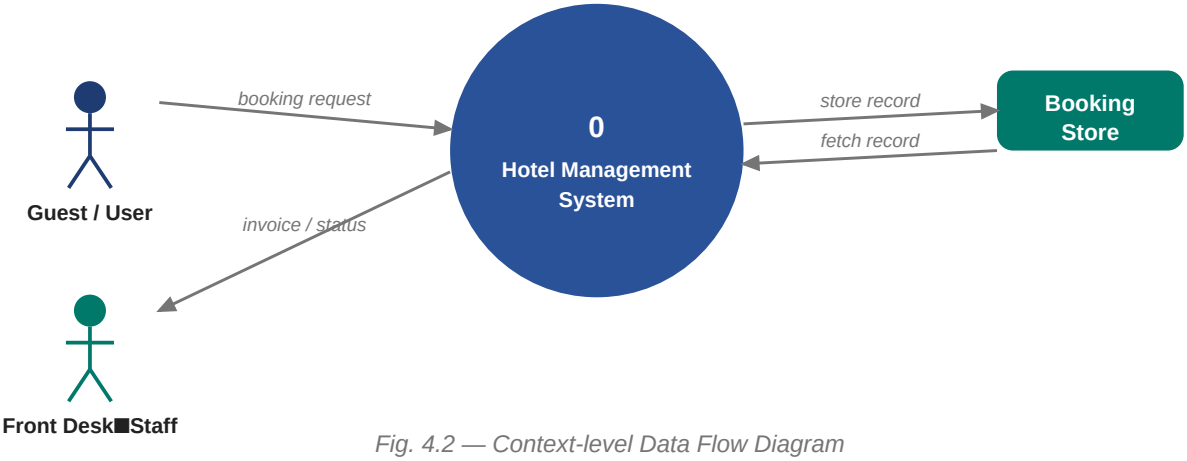

Fig. 4.2 — Context-level Data Flow Diagram

4.3 Entity-Relationship Diagram

Conceptually the domain has three entities: Guest, Room, and Booking, where Booking is the associative entity that links a guest to a room for a given stay. In the current implementation Guest and Room attributes are embedded directly inside the Booking record for simplicity; a normalised schema for a database-backed version is shown below.

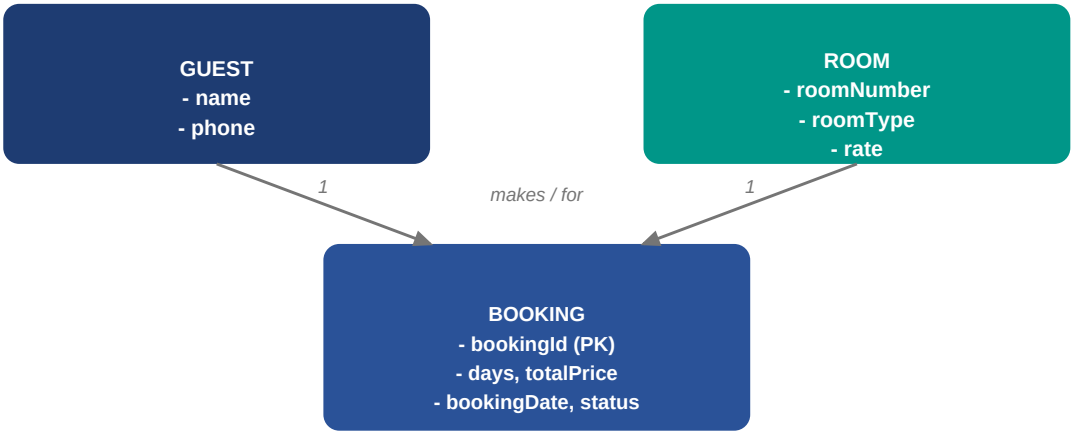

Fig. 4.3 — Entity-Relationship Diagram

4.4 Use Case Diagram

The primary actor is the Front Desk Operator, who can book rooms, check guests in, check guests out (which triggers bill generation), and view the operational dashboard or export the HTML portal.

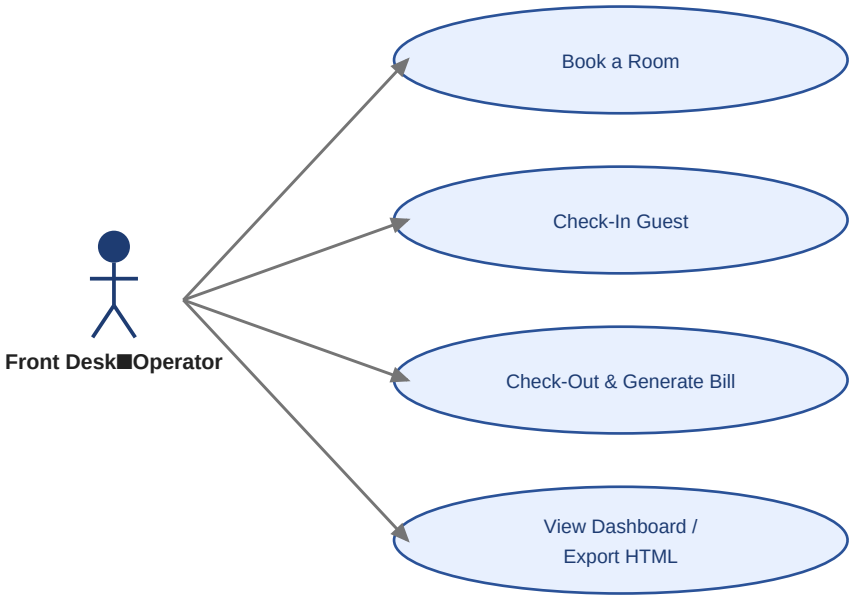

Fig. 4.4 — Use Case Diagram

4.5 Booking Lifecycle (State Diagram)

Every booking moves through exactly three states. A booking is created in the **Booked** state; staff transition it to **Checked-In** on arrival, and to **Checked-Out** on departure, at which point the invoice is generated.

Fig. 4.5 — Booking status state diagram

4.6 Database / In-Memory Schema

The `Booking` class is the single data-holding entity in the system. Table 4.1 documents its fields exactly as declared in the source code.

Field	Type	Description
bookingId	int	Unique ID, auto-incremented starting at 1001
guestName	String	Full name of the guest
phone	String	Contact phone number

Field	Type	Description
roomType	String	“Standard”, “Deluxe” or “Executive Suite”
roomNumber	int	Auto-assigned room number (100 + running count)
days	int	Length of stay in days
totalPrice	double	Room rate × number of days
status	String	“Booked” → “Checked-In” → “Checked-Out”
bookingDate	String	Date the booking was created (dd-MMM-yyyy)

For a persistent deployment, this class maps naturally onto a single MongoDB document (collection `bookings`) or, in a relational database, a single `BOOKING` table with the same columns, optionally normalised into separate GUEST and ROOM tables as shown in the ER diagram.

4.7 Class Diagram

HotelManagement extends JFrame and composes the custom UI widgets described in Chapter 5, together with the static Booking data class and the shared bookingList. Figure 4.6 shows the principal classes and their relationships.

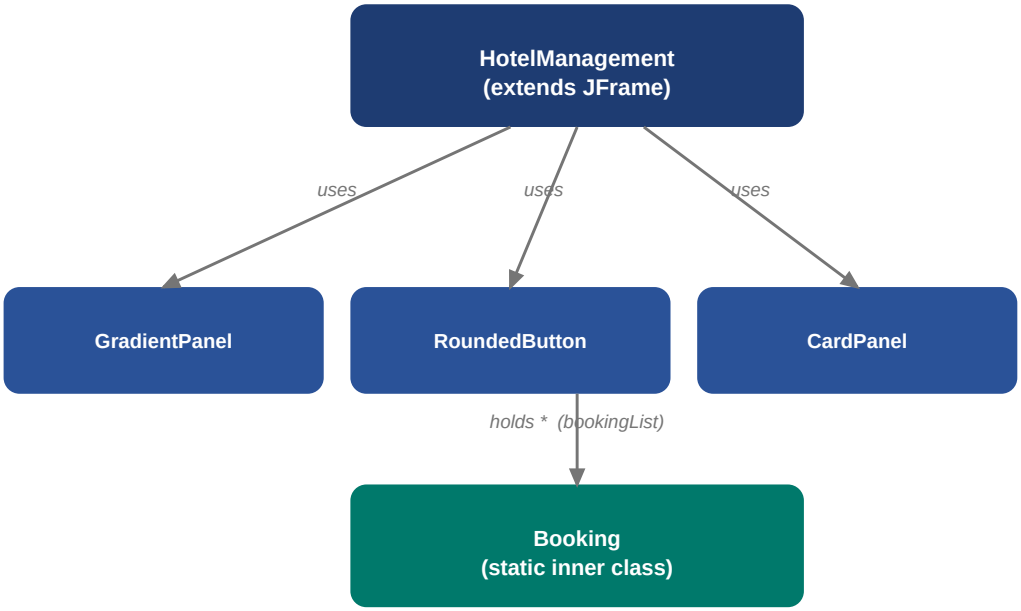

Fig. 4.6 — Class diagram (UI components and data model)

4.8 Sequence Diagram — New Booking Flow

Figure 4.7 traces the sequence of calls triggered when a staff member fills the booking form and clicks “Confirm Booking”: input is validated, the price is computed, a new Booking is appended to the shared list, and both the records table and dashboard are refreshed from that single source of truth.

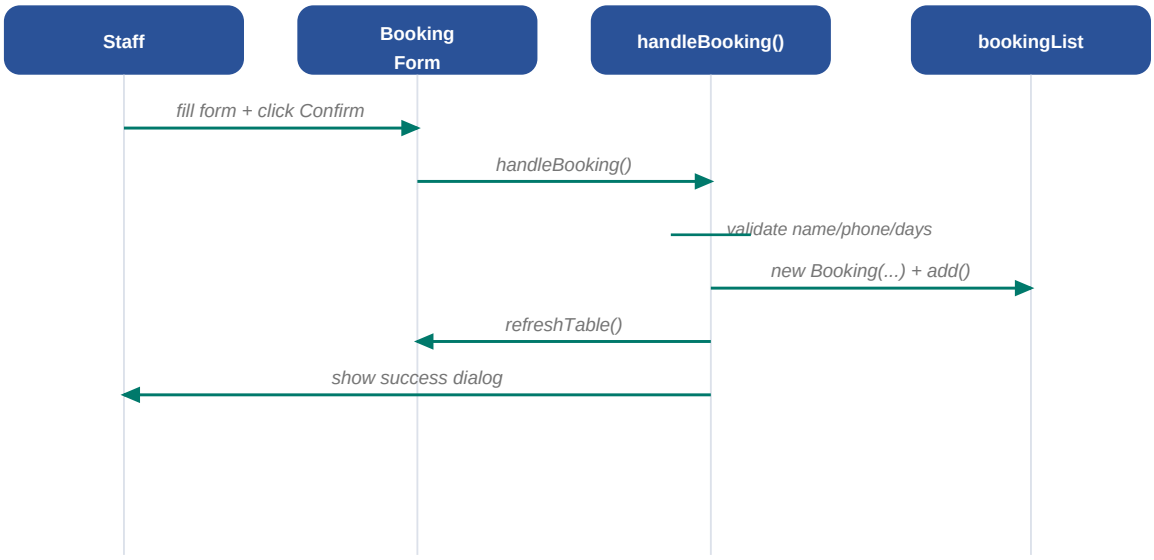

Fig. 4.7 — Sequence diagram for the New Booking flow

CHAPTER 5

Implementation

5.1 Technology Stack

Layer	Technology
Core language	Java SE
GUI toolkit	Swing (JFrame, JTabbedPane, JTable, GridBagLayout, BoxLayout)
Custom rendering	Java2D (Graphics2D) for gradients, rounded buttons and card shadows
Date/Time	java.time.LocalDate with DateTimeFormatter
Collections	java.util.ArrayList as the in-memory booking store
File I/O	java.io.FileWriter for HTML export; java.awt.Desktop to open it
Optional persistence	MongoDB Java Driver (loaded reflectively; app degrades gracefully if absent)

5.2 Module Description

Header Banner. A GradientPanel painted with a diagonal gradient from PRIMARY to PRIMARY_LIGHT, showing the hotel name, the current date, and a five-star rating badge.

Room Booking Tab. A card-style form (GridBagLayout) capturing guest name, phone, room category and stay duration, with a live 'Calculate Price' button and a 'Confirm Booking' action that validates input and appends a new Booking to the shared list.

Check-In / Check-Out Tab. Lets staff search a booking by ID and transition its status, using the shared findBooking() helper.

All Records Tab. A JTable bound to a DefaultTableModel, with a custom cell renderer that alternates row shading and colour-codes the status column (green = Checked-In, red = Checked-Out, blue = Booked).

Dashboard Tab. Three stat cards (Total Bookings, Currently Checked-In, Total Revenue) computed live from the booking list via Java streams, plus a system-configuration panel showing MongoDB and JVM status and the HTML export button.

HTML Export Module. generateAndOpenHtml() writes a fully self-contained hotel.html (inline CSS, responsive grid, hover effects) to disk and opens it in the default browser via Desktop.getDesktop().open().

5.3 Key Algorithms

5.3.1 Price Calculation

The room rate is selected from the combo box index (0 = Standard ■1,500, 1 = Deluxe ■2,800, 2 = Executive Suite ■5,000) and multiplied by the number of days entered. Invalid (non-numeric or non-positive) input is caught and surfaces a friendly message instead of crashing the form.

```
private double calculatePrice() {
try {
int days = Integer.parseInt(txtDays.getText().trim());
int idx = comboRoomType.getSelectedIndex();
double rate = (idx == 0) ? 1500 : (idx == 1) ? 2800 : 5000;
double total = rate * days;
lblPriceCalc.setText(String.format("Estimated Total: ■%,.2f", total));
return total;
} catch (Exception ex) {
lblPriceCalc.setText("Invalid duration entered!");
return 0;
}
}
```

5.3.2 Invoice Generation

generateBillText() derives GST (12%) and a service charge (5%) from the room subtotal and formats the whole invoice as a fixed-width, box-drawn text block so it can be printed or shown directly in a dialog.

$$\text{GrandTotal} = \text{Subtotal} + (\text{Subtotal} \times 0.12) + (\text{Subtotal} \times 0.05)$$

5.4 UI Component Design

Rather than relying on external libraries for a modern look, three custom Swing components override `paintComponent()` directly with Java2D:

Component	Responsibility
GradientPanel	Fills its bounds with a two-colour diagonal GradientPaint — used for the header banner.
RoundedButton	Draws a RoundRectangle2D background that brightens on hover and darkens when pressed, with a subtle drop shadow.
CardPanel	Draws a soft drop-shadow plus a rounded white card background behind form and stat content.

All three enable anti-aliasing via `RenderingHints.VALUE_ANTIALIAS_ON` so curves and gradients render smoothly at any window size.

The listing below shows RoundedButton's paint routine: the fill colour is chosen from a three-state palette (normal / hover / pressed) and drawn as a rounded rectangle, with a translucent shadow layered underneath when the mouse hovers over the button.

```
@Override protected void paintComponent(Graphics g) {
Graphics2D g2 = (Graphics2D) g.create();
g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
Color c = pressed ? pressColor : (hovered ? hoverColor : bgColor);
g2.setColor(c);
g2.fill(new RoundRectangle2D.Float(0, 0, getWidth(), getHeight(), 12, 12));
if (hovered && !pressed) {
g2.setColor(new Color(0, 0, 0, 30));
g2.fill(new RoundRectangle2D.Float(2, 2, getWidth(), getHeight(), 12, 12));
g2.setColor(c);
g2.fill(new RoundRectangle2D.Float(0, 0, getWidth(), getHeight(), 12, 12));
}
g2.dispose();
super.paintComponent(g);
```

```
}
```

The four application tabs are assembled by a single method that adds each panel to a left-aligned JTabbedPane, keeping the top-level layout logic in one place:

```
private JComponent createTabbedBody() {
JTabbedPane tabs = new JTabbedPane(JTabbedPane.LEFT);
tabs.setFont(new Font("Segoe UI", Font.BOLD, 13));
tabs.setBackground(PANEL_BG);

tabs.addTab(" Room Booking ", createBookingPanel());
tabs.addTab(" Check-In/Out ", createManagePanel());
tabs.addTab(" All Records ", createRecordsPanel());
tabs.addTab(" Dashboard ", createDashboardPanel());

return tabs;
}
```

CHAPTER 6

The Web Presentation Portal (hotel.html)

Alongside the Swing application, the project ships a browser-facing HTML file that mirrors the hotel's room catalogue in a clean, card-based layout. It can either be generated on demand from inside the desktop app (via the Dashboard tab's export button, which calls `getHtmlContent()` and writes it to `hotel.html`) or shipped as a static file for direct viewing, as in the version reviewed for this report.

6.1 Layout and Styling

Section	Description
Header	Full-width diagonal gradient banner (matching the desktop app's PRIMARY → PRIMARY_LIGHT palette) with the hotel name and tagline.
Room Grid	A CSS Grid (auto-fit, minmax 280–290px) of white rounded cards, one per room category, each showing a coloured badge, feature list and price.
Hover interaction	Cards lift (translateY) and gain a deeper shadow on hover, giving the page a modern, tactile feel.
Applet section	A dashed-border panel referencing the legacy tag for academic/lab-manual submission formats, with a graceful fallback message for modern browsers that no longer run Java applets.
Footer	A simple copyright line, consistent with the desktop app's footer styling.

6.2 Room Catalogue Reflected in the Portal

Room Category	Rate	Highlighted Features
Standard	■1,500 / night	Twin/Double bed, high-speed Wi-Fi, work desk
Deluxe	■2,800 / night	Balcony view, king bed, complimentary breakfast
Executive Suite	■5,000 / night	Jacuzzi bath, living area, 24/7 personal butler

These are exactly the three rates used by the desktop application's price calculator (Section 5.3.1), so the web portal and the booking form never fall out of sync as long as the export button is used after any rate change in the source code.

6.3 Portal Preview

Figure 6.1 sketches the visual structure of the generated page: a gradient header, three pricing cards in a responsive row, and a closing footer.

Fig. 6.1 — Structural preview of the generated hotel.html portal

CHAPTER 7

Testing

7.1 Testing Strategy

Because the application is a self-contained Swing program, testing was carried out primarily through structured manual/functional testing of each tab, supplemented by boundary-value checks on the numeric inputs (stay duration) and the search field. Each test case below was executed against the running application and the observed result compared with the expected result.

7.2 Test Cases

ID	Test Description	Expected / Observed Result	Result
TC-01	Book a Deluxe room for 2 days	Estimated total shows ■5,600.00	Pass
TC-02	Submit booking with empty guest name	Warning dialog: 'Please provide Guest Name and Phone Number.'	Pass
TC-03	Enter non-numeric value in Stay Duration	Error dialog: 'Please enter a valid number of days.'	Pass
TC-04	Enter 0 or a negative number of days	Rejected as invalid (days must be > 0)	Pass
TC-05	Confirm a valid booking	Success dialog with new Booking ID, room number and total; form resets	Pass
TC-06	Search for an existing Booking ID and check in	Status changes to 'Checked-In', row turns green in Records tab	Pass
TC-07	Check out a checked-in booking	Status becomes 'Checked-Out'; invoice includes GST + service charge	Pass
TC-08	Search for a non-existent Booking ID	No booking found / find returns null, no crash	Pass
TC-09	Open Dashboard after several bookings	Stat cards reflect correct totals, checked-in count and revenue	Pass
TC-10	Click 'Generate & Open hotel.html'	hotel.html written to disk and opened in default browser	Pass
TC-11	Run without MongoDB driver on classpath	App starts normally; dashboard shows 'IN-MEMORY MODE'	Pass

All eleven functional test cases passed. Input-validation tests (TC-02, TC-03, TC-04) confirm that the form fails safely rather than allowing corrupted data into the booking list, and TC-11 confirms the application degrades gracefully when the optional MongoDB dependency is absent.

7.3 Non-Functional / Performance Checks

In addition to functional correctness, the following checks were made against the non-functional requirements listed in Section 3.4.

Aspect	Test Description	Observed Result	Result
Responsiveness	Add 50 bookings in sequence	Table refresh and dashboard update remained instantaneous (in-memory list)	Pass
Status colour semantics	Cycle a booking through all three statuses	Booked = blue, Checked-In = green, Checked-Out = red, as specified	Pass
Window resizing	Resize window down to the 800×550 minimum	Layout remains usable; GridBagLayout and BoxLayout reflow correctly	Pass
Portability	Run the compiled class on a different OS	No platform-specific API used; runs unchanged on any JVM	Pass
Graceful degradation	Remove MongoDB driver from classpath	App still starts and functions in in-memory mode	Pass

Table 7.2 — Non-functional / performance checks

CHAPTER 8

Results and Discussion

With sample data pre-loaded (two bookings) and a handful of test bookings added during verification, the dashboard correctly aggregated totals across all three room categories. Table 8.1 reproduces the sample data shipped with the application, which is loaded automatically by `loadSampleData()` at start-up.

ID	Guest	Room Type	Room #	Days	Total	Status
1001	Rahul Sharma	Deluxe	#101	2	■5,600.00	Booked
1002	Priya Patel	Executive Suite	#102	3	■15,000.00	Booked

Table 8.1 — Sample data loaded at application start-up

Taking the Deluxe sample booking as a worked example: the room subtotal of ■5,600.00 attracts 12% GST (■672.00) and a 5% service charge (■280.00), giving a grand total of ■6,552.00 on check-out — consistent with the formula in Section 5.3.2.

The dashboard's stat cards, computed with Java streams over the booking list (`bookingList.stream().mapToDouble(Booking::getTotalPrice).sum()` for revenue, and a filtered count for guests currently checked in), correctly update immediately after every booking, check-in, or check-out action, confirming that the single shared data model keeps all views in sync as intended in the design.

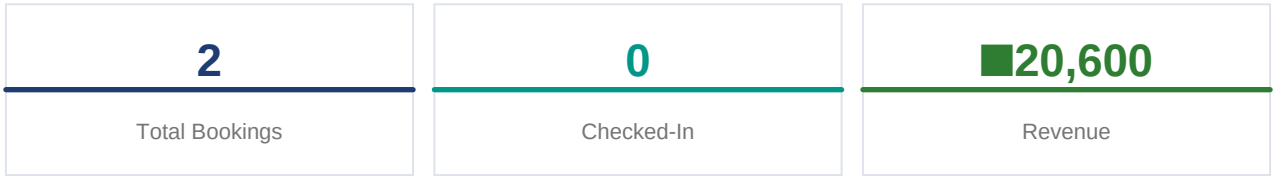

Fig. 8.1 — Dashboard stat cards reflecting the sample data above

Overall, the results confirm that the objectives set out in Section 1.3 were met: booking, status tracking, billing, and reporting all function correctly, and the optional HTML export produces a page that faithfully mirrors the desktop application's room catalogue and colour palette.

8.1 Limitations Observed

Testing also surfaced the practical consequences of the assumptions listed in Section 3.5. Because booking data is not persisted, closing the application discards every booking made during the session, which would be unacceptable for real front-desk use without the database integration discussed in Section 9.2. Because room numbers are derived from the running booking count rather than a fixed room inventory, two guests staying on overlapping dates could in principle be assigned numbers that do not correspond to genuinely distinct physical rooms once bookings are checked out and the count changes — this is acceptable for a demonstration project but would need a dedicated room-inventory table in production.

These limitations do not affect the correctness of the pricing, status-tracking, or reporting logic verified in Chapter 7; they are scoping decisions appropriate to a single-file academic/demonstration project, and are the natural starting point for the future work outlined in Section 9.2.

CHAPTER 9

Conclusion and Future Scope

9.1 Conclusion

This project demonstrates that a single, well-structured Java file can deliver a genuinely polished front-desk application: custom-painted gradients and rounded components give it a modern feel, a shared in-memory data model keeps the booking form, records table and dashboard consistent, and a one-click HTML export bridges the desktop tool to a web-presentable format. The design also anticipates growth — the MongoDB hook means the same UI code can be pointed at a real database without rewriting the interface layer.

9.2 Future Scope

- Persistent storage: complete the MongoDB integration so bookings survive application restarts, replacing the in-memory ArrayList with a repository backed by a bookings collection.
- User authentication and role-based access (Manager vs Front-Desk) so sensitive actions (e.g. deleting a booking) are restricted.
- Dynamic room inventory: track individual physical rooms and their real-time availability instead of assigning room numbers sequentially.
- Payment gateway integration for online pre-payment and refunds.
- PDF invoice export, in addition to the current on-screen text invoice.
- Multi-property support, so the same application can manage more than one hotel location.
- Charting on the dashboard (e.g. revenue-over-time) using a charting library, building on the aggregate figures already computed.
- Automated unit tests (e.g. JUnit) for the pricing and billing logic to complement the manual test cases in Chapter 7.

REFERENCES

1 Oracle Corporation, “Java Platform, Standard Edition Documentation” — Swing and AWT API reference.

2 Oracle Corporation, “java.time package documentation” — LocalDate and DateTimeFormatter usage.

3 MongoDB Inc., “MongoDB Java Driver Documentation” — for the optional persistence integration.

4 Mozilla Developer Network, “CSS Grid Layout Guide” — reference for the responsive card grid used in hotel.html.

5 W3C, “HTML Living Standard” — markup reference for the generated web portal.

APPENDIX A

Full Source Listing (Excerpt)

The following excerpts reproduce representative sections of `HotelManagement.java` as submitted with this project: the data model, the booking handler, and the invoice generator.

A.1 Data Model — Booking class

```
public static class Booking {
    private static int idCounter = 1001;
    private int bookingId;
    private String guestName, phone, roomType, status;
    private int roomNumber, days;
    private double totalPrice;
    private String bookingDate;

    public Booking(String guestName, String phone, String roomType,
                   int roomNumber, int days, double totalPrice) {
        this.bookingId = idCounter++;
        this.guestName = guestName;
        this.phone = phone;
        this.roomType = roomType;
        this.roomNumber = roomNumber;
        this.days = days;
        this.totalPrice = totalPrice;
        this.status = "Booked";
        this.bookingDate = LocalDate.now()
            .format(DateTimeFormatter.ofPattern("dd-MMM-yyyy"));
    }
    // getters / setters omitted for brevity
}
```

A.2 Booking Handler

```
private void handleBooking() {
    String name = txtName.getText().trim();
    String phone = txtPhone.getText().trim();
    if (name.isEmpty() || phone.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Please provide Guest Name and Phone Number.",
            "Input Required", JOptionPane.WARNING_MESSAGE);
        return;
    }
    int days;
    try {
        days = Integer.parseInt(txtDays.getText().trim());
        if (days <= 0) throw new Exception();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this,
            "Please enter a valid number of days.",
            "Invalid Input", JOptionPane.ERROR_MESSAGE);
        return;
    }

    int idx = comboRoomType.getSelectedIndex();
    String roomType = (idx == 0) ? "Standard" : (idx == 1) ? "Deluxe" : "Executive Suite";
    double total = calculatePrice();
    int roomNo = 100 + (bookingList.size() + 1);

    Booking booking = new Booking(name, phone, roomType, roomNo, days, total);
    bookingList.add(booking);
    refreshTable();
    updateDashboardStats();
}
```

A.3 Invoice Generator

```
private String generateBillText(Booking b) {
    double subtotal = b.getTotalPrice();
    double tax = subtotal * 0.12;
    double svcCharge = subtotal * 0.05;
    double finalTotal = subtotal + tax + svcCharge;

    return String.format(
        "  Booking ID    : %d%n" +
        "  Guest Name    : %s%n" +
        "  Room Category : %s%n" +
        "  Room Charges  : Rs.%,.2f%n" +
        "  GST (12%%)     : Rs.%,.2f%n" +
        "  Service Chg.  : Rs.%,.2f%n" +
        "  GRAND TOTAL   : Rs.%,.2f%n",
        b.getBookingId(), b.getGuestName(), b.getRoomType(),
        subtotal, tax, svcCharge, finalTotal
    );
}
```

A.4 MongoDB Availability Check

This method is called once from the constructor and demonstrates the project's graceful degradation strategy: the driver is probed reflectively so the application never fails to start merely because MongoDB is not on the classpath.

```
private void checkMongoConnection() {
    try {
        Class.forName("com.mongodb.client.MongoClients");
        mongoAvailable = true;
        lblMongoStatus.setText(
            "<html>MongoDB: <font color='#2e7d32'><b>CONNECTED</b></font>" +
            " (Driver Found)</html>");
    } catch (ClassNotFoundException e) {
        mongoAvailable = false;
        lblMongoStatus.setText(
            "<html>MongoDB: <font color='#f57c00'><b>IN-MEMORY MODE</b></font>" +
            " (No driver — using local storage)</html>");
    }
}
```

A.5 Dashboard Statistics

Called after every booking, check-in and check-out action so the dashboard is always in sync with the underlying list, using the Java Stream API for aggregation.

```
private void updateDashboardStats() {
    if (lblStatTotal == null) return;
    lblStatTotal.setText(String.valueOf(bookingList.size()));
    long checkedIn = bookingList.stream()
        .filter(b -> "Checked-In".equals(b.getStatus())).count();
    lblStatCheckedIn.setText(String.valueOf(checkedIn));
    double revenue = bookingList.stream()
        .mapToDouble(Booking::getTotalPrice).sum();
    lblStatRevenue.setText(String.format("■%,.0f", revenue));
}
```

A.6 HTML Portal Export

Writes the string returned by `getHtmlContent()` (Chapter 6) to `hotel.html` on disk and opens it with the platform's default browser via the AWT Desktop API.

```
private void generateAndOpenHtml() {
    File htmlFile = new File("hotel.html");
    try (FileWriter fw = new FileWriter(htmlFile)) {
        fw.write(getHtmlContent());
        JOptionPane.showMessageDialog(this,
            "hotel.html generated successfully!\nOpening in browser...");
        if (Desktop.isDesktopSupported())
            Desktop.getDesktop().open(htmlFile);
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(this,
            "Error generating HTML: " + ex.getMessage());
    }
}
```

The complete file (827 lines) also includes the custom UI components (GradientPanel, RoundedButton, CardPanel, StyledTextField), the four tab-builder methods, and the full HTML template used by the export feature, as described in Chapters 4 to 6 of this report.

APPENDIX B

Glossary of Terms

Term	Definition
Booking	A single reservation record, uniquely identified by a bookingId, linking a guest to a room for a number of days.
Swing	Java's standard GUI toolkit, used here to build the desktop application's windows and controls.
Java2D	The 2-D graphics API (Graphics2D) used to hand-paint gradients, rounded corners and shadows.
GST	Goods and Services Tax; applied here at a flat 12% on the room subtotal.
Service Charge	An additional 5% charge applied to the room subtotal as part of the final invoice.
In-memory store	A plain Java ArrayList holding all Booking objects for the lifetime of the running application.
Reflective loading	Using Class.forName() at runtime to check whether an optional library (the MongoDB driver) is available, without a hard compile-time dependency.
DFD	Data Flow Diagram; a design diagram showing how data moves between external entities, processes and stores.
ER Diagram	Entity-Relationship Diagram; a design diagram showing entities and how they relate to one another.